

ABSTRACT

AirSketch is an innovative gesture-based virtual drawing application designed to transform digital drawing by using real-time hand gesture recognition through a webcam. Unlike traditional drawing systems that rely on devices such as a mouse, stylus, or touchscreen, the application enables users to interact with a virtual canvas using natural hand and finger movements. By extending a single finger, users can draw seamlessly, while predefined gestures allow them to change colors, activate eraser mode, clear the canvas, and control drawing functions efficiently. The system uses computer vision and real-time hand tracking techniques to accurately detect hand landmarks and convert gestures into precise drawing actions with minimal latency. The project integrates technologies such as computer vision, image processing, and human-computer interaction to provide a smooth, touch-free, and user-friendly experience. The salient features of AirSketch include real-time gesture detection, virtual canvas interaction, multiple color selection, eraser functionality, touchless operation, smooth and responsive drawing, and accessibility support for physically challenged users. The application enhances creativity and user engagement while demonstrating the practical implementation of gesture-controlled systems in modern web technologies. Furthermore, AirSketch highlights the growing importance of contactless and intelligent interaction systems in areas such as education, entertainment, design, and smart digital environments, making digital drawing more interactive, efficient, and inclusive.

CONTENTS

1. INTRODUCTION	3
1.1 Domain Introduction	3
1.2 Project Introduction	4
2. LITERATURE SURVEY	5
3. EXISTING AND PROPOSED SYSTEM	21
4. PROBLEM DEFINITION, OBJECTIVES AND PROPOSED METHODOLOGY	24
5. SYSTEM DESIGN	28
5.1 Block Diagram of project	28
5.2 Module Description	29
6. SYSTEM IMPLEMENTATION	32
7. EXPERIMENTAL RESULTS AND DISCUSSION	38
8. CONCLUSION AND FUTURE SCOPE	44
REFERENCES	46

I. INTRODUCTION

1.1 Domain Introduction

Human–computer interaction has progressively moved from keyboard- and mouse-centred interaction toward interfaces that interpret more natural forms of input. Hand gestures are particularly attractive because they allow pointing, selection, manipulation, and continuous movement without a physical controller. Gesture-driven interaction appears in virtual reality, gaming, assistive technology, air-writing, robotics, and creative applications. The literature supplied with this project shows a broad range of approaches, from wearable acoustic-optic sensing to webcam-based landmark tracking and learned temporal gesture models.

Computer vision provides a practical route to touchless interaction because a camera can observe hand movement without requiring the user to wear a sensor. A vision pipeline typically contains image acquisition, hand detection or segmentation, landmark or feature extraction, gesture interpretation, and an application-specific response. The reliability of the final interaction depends not only on the hand-tracking model but also on lighting, camera quality, background conditions, hand occlusion, interaction rules, and the way continuous motion is converted into commands.

AirSketch occupies the application-level end of this design space. It does not attempt to introduce a new hand detector or train a general-purpose gesture classifier. Instead, it uses the established MediaPipe hand-landmark representation and adds deterministic geometric rules for a virtual drawing interface. This makes the interaction logic easy to inspect: the system knows where the fingertip is, measures the distance between two fingertips, and uses that distance to decide whether the user is actively drawing.

.

1.2 Project Introduction

AirSketch receives a live 1280×720 camera stream. The frame is horizontally flipped to provide a mirror-like interaction view and passed to MediaPipe Hands. When a hand is detected, landmark 8, corresponding to the index fingertip, is used as the pointer position. Landmark 12, the middle-finger tip, is used together with landmark 8 to calculate a Euclidean distance. A distance below 70 pixels is interpreted as a pinch and activates the drawing state.

The pointer coordinates are smoothed using the arithmetic mean of the most recent five valid fingertip positions. This reduces small visual jitter without introducing a computationally expensive filter. The system distinguishes the toolbar region from the drawing region. When the smoothed pointer is above the toolbar boundary, it is interpreted as UI input. Otherwise, an active pinch starts or continues drawing. Brush and eraser tools draw continuously between successive points, while geometric tools use a temporary preview and commit the final shape when the pinch is released.

1.3 Scope of the Project

- The project covers real-time single-hand tracking from a conventional RGB webcam.
- The project covers deterministic pinch-based activation and fingertip smoothing.
- The project covers freehand brush, eraser, line, rectangle, circle, filled rectangle, filled circle, and clear-canvas tools.
- The project covers colour selection and ten predefined brush thickness values from 5 to 50 pixels.
- The project covers frame-level evaluation of the pinch detector using labelled samples and standard classification metrics.

II. LITERATURE SURVEY

The literature reviewed for AirSketch spans traditional computer-vision gesture recognition, webcam-based air drawing, MediaPipe-based interfaces, multimodal depth-aware recognition, temporal deep-learning approaches, assistive interaction, and software reliability. The current research paper uses sixteen references, while the earlier project report contains a ten-item survey. The following discussion consolidates the supplied material and relates each group of work to the design decisions made in AirSketch.

[1] AO-Finger: Hands-free Fine-grained Finger Gesture Recognition via Acoustic-Optic Sensor Fusing, Chenhan Xu, Bing Zhou, Gurunandan Krishnan, and Shree Nayar, 2023

Xu et al. proposed **AO-Finger**, a hands-free fine-grained finger gesture recognition system designed for wearable and Extended Reality (XR) interaction. The system combines acoustic and optical sensing to recognize natural finger movements without requiring users to hold a controller. It uses two optical sensors to track thumb movement and a modified microphone to capture acoustic signals generated by finger movements. These complementary sensing modalities help distinguish gestures that may produce similar acoustic signals when used alone. The system recognizes three fast gestures—**flick, pinch, and tap**—and also supports continuous finger swipe tracking. A multimodal **CNN-Transformer** model is used for fast gesture recognition, while a separate model is used for detecting finger contact during swipe interactions.

The prototype consists of a wrist-worn structure containing optical sensors, a modified MEMS microphone, and a Bluetooth Low Energy controller. The optical sensing system operates at approximately **386 FPS**, enabling rapid finger movements to be captured. Experimental evaluation achieved an overall accuracy of **94.83%** for fast gesture recognition. The authors also discussed limitations such as partial dependence on external computation, the need for relatively static hand positioning during gesture detection, and the need for larger-scale user evaluation.

The work is relevant to **AirSketch** because both systems use finger gestures, particularly **pinch**, for natural human-computer interaction. However, AirSketch uses a conventional camera and **MediaPipe hand landmarks** instead of specialized acoustic and optical wrist hardware. In AirSketch, fingertip positions and pinch detection are used to control virtual drawing, shapes, and other canvas operations. Therefore, AO-Finger demonstrates the potential of real-time finger gestures for hands-free interaction, while AirSketch applies a camera-based approach to create a **gesture-controlled virtual drawing system**.

The study also highlights the importance of **low-latency and reliable gesture recognition** for interactive applications. This is particularly relevant to AirSketch, where continuous hand tracking and quick gesture detection are required to provide a smooth drawing experience. The evaluation methodology of AO-Finger also provides useful guidance for assessing gesture recognition performance using measures such as **accuracy, precision, recall, and F1-score**.

[2] OMG-Bench: A New Challenging Benchmark for Skeleton-based Online Micro Hand Gesture Recognition, Haochen Chang, Pengfei Ren, Buyuan Zhang, Da Li, Tianhao Han, Haoyang Zhang, Liang Xie, Hongbo Chen, and Erwei Yin, 2026

Chang et al. introduced **OMG-Bench**, a large-scale benchmark for online micro hand gesture recognition using hand skeleton data. The work addresses the difficulty of recognizing subtle, rapid, and continuously performed gestures, where different gestures can have very similar movements and gesture boundaries may be difficult to determine. The proposed dataset contains **40 fine-grained gesture classes, 1,272 sequences, and 13,948 gesture instances**, including interactions such as tap, double-tap, swipe, pinch, and release. The dataset was created using a synchronized **five-camera**

RGB-D system, followed by multi-view hand pose estimation and semi-automatic frame-level annotation. The resulting skeletons contain 21 hand joints, with the authors reporting a mean per-joint position error of **2.78 mm** on their evaluation set.

To address the challenges of online recognition, the authors proposed **Hierarchical Memory-Augmented Transformer (HMATr)**, an end-to-end framework that performs gesture detection and classification together. The method uses an **ST-GCN backbone** to extract skeleton features, followed by frame-level and window-level memory banks that preserve information from previous time windows. Position-aware learnable queries are then used to capture the location and semantic information of gestures. This approach is particularly useful for continuous gesture streams because it reduces the loss of temporal information that can occur when independent sliding windows are processed. On OMG-Bench, HMATr achieved a **89.2% detection rate**, with a **0.22 false-positive score**, and also showed lower inference delay than the compared methods. The authors' ablation study further showed that removing the memory components or position-aware queries reduced performance, demonstrating their contribution to online recognition.

The study is relevant to **AirSketch** because both systems rely on **hand-skeleton/landmark information and real-time gesture interaction**. AirSketch currently uses MediaPipe hand landmarks and geometric distance between the index and middle fingertips to detect a pinch gesture, whereas OMG-Bench focuses on recognizing a much wider range of subtle gestures from continuous skeleton sequences.

[3] Air Writing App with Real-Time Hand Tracking and Handwriting Recognition using Fingertip Detection and Drawing, Heena and Dr. Sandeep Ranjan, 2023

Heena and Dr. Sandeep Ranjan proposed an air-writing application that enables users to write, draw, and erase in a virtual canvas using **real-time hand tracking and fingertip detection**. The system uses **OpenCV and MediaPipe** to process webcam video and identify 21 hand landmarks, including the fingertip. The detected fingertip coordinates are continuously tracked to interpret hand movements and convert them into drawing actions. The authors use **Euclidean distance-based gesture recognition** to distinguish actions such as drawing and erasing. The application also

provides features such as color selection, adjustable line thickness, brush size, and switching between drawing and erasing modes.

The study additionally focuses on **handwriting recognition** after the user's writing has been captured on the virtual canvas. Several preprocessing techniques, including auto-orientation, auto-contrast adjustment, adaptive histogram equalization, and grayscale conversion, are applied to improve recognition under different conditions. The paper evaluates the handwriting recognition system using accuracy, precision, recall, and F1-score. It reports an overall accuracy of **86.36%**, with a precision of **90.91%**, recall of **83.33%**, and F1-score of **86.96%** for the reported drawing class. The authors also discuss challenges such as variations in lighting, handwriting styles, hand speed, and user posture, which can affect real-world performance.

This work is highly relevant to **AirSketch** because both systems use **webcam-based hand tracking, fingertip movement, and a virtual canvas** to provide non-contact drawing interaction. However, AirSketch extends this concept by using gesture-based controls for functions such as **brush, eraser, colors, and geometric shapes**, rather than focusing mainly on handwriting recognition. The paper therefore provides a strong foundation for AirSketch's use of MediaPipe hand landmarks and fingertip tracking, while also highlighting the importance of handling environmental and user-related variations for reliable real-time interaction.

[4] AIR CANVAS HAND TRACKING, Swathi. A, 2024

Swathi. A proposed an **Air Canvas Hand Tracking** system that allows users to draw and interact with a virtual canvas using real-time hand gestures. The system uses a standard webcam along with **OpenCV and MediaPipe** to capture and process hand movements. MediaPipe detects hand landmarks and provides 21 joint coordinates, which are used to identify finger positions and interpret gestures. The proposed system supports drawing, color selection, brush-size adjustment, and other canvas operations without requiring physical contact with an input device. The paper

emphasizes real-time processing, intuitive interaction, portability, and low cost because it relies on commonly available hardware and open-source libraries.

The system processes webcam frames by converting the captured images from BGR to RGB and identifying the position of the fingers through MediaPipe. The index fingertip is primarily used for drawing, while different finger gestures can be used for selecting shapes, colors, and tools. The system also includes functionality for saving the completed canvas as an image. Its design focuses on providing immediate visual feedback as the user's gestures are converted into digital drawing commands. The paper identifies environmental factors such as **lighting, clutter, and hand occlusion** as challenges that can affect gesture recognition.

This work is highly relevant to **AirSketch** because both systems use **camera-based hand tracking, MediaPipe landmarks, fingertip movement, and a virtual canvas** for non-contact drawing. The proposed Air Canvas system is particularly similar to AirSketch in its use of real-time finger tracking and gesture-based controls for drawing, colors, and brush settings. However, AirSketch extends this concept by incorporating **pinch-based interaction, multiple drawing tools, geometric shapes, and gesture evaluation**.

The paper therefore supports the use of MediaPipe and real-time hand landmarks as an effective foundation for gesture-controlled drawing systems, while also highlighting environmental factors that should be considered when improving AirSketch's reliability.

[5] A Deep Learning-based Multimodal Depth-Aware Dynamic Hand Gesture Recognition System, Hasan Mahmud, Mashrur M. Morshed, and Md. Kamrul Hasan, 2021

Mahmud et al. proposed a **deep learning-based multimodal system for dynamic hand gesture recognition** using depth information and 2D hand-skeleton coordinates. The authors identified that raw depth images often have low contrast in hand regions, making fine details such as finger orientation, finger overlap, and palm structure difficult to distinguish. To address this, they

introduced **Grayscale Variation (gVar)**, a preprocessing technique that quantizes continuous depth values into discrete gray levels, producing sharper contrast between important regions of the hand. The processed depth information is then combined with 2D skeleton coordinates to provide complementary spatial and temporal information for gesture recognition.

The proposed architecture is a **feature-level multimodal fusion CRNN** consisting of two main branches. The first branch uses a CNN followed by a bidirectional LSTM to extract spatial and temporal features from depth-image sequences, while the second uses an LSTM to process 2D skeleton joint sequences. Their extracted features are concatenated and passed to a classification network. The authors also reduce the depth-image resolution to **50×50** and introduce Batch Normalization, dropout, and other regularization techniques to reduce overfitting and model complexity. The proposed model contains approximately **4.4 million trainable parameters**, compared with more than 31 million in the previous fusion approaches.

The system was evaluated on the **DHG-14/28 and SHREC-2017 datasets**. On DHG-14/28, the proposed method achieved **90.21% accuracy for 14 gestures and 88.64% for 28 gestures**, while achieving **93.33% and 90.24%** respectively on SHREC-2017. The study also showed that gVar improved recognition, particularly for fine-grained gestures, although some confusion remained between visually similar gestures such as **grab and pinch**. This work is relevant to **AirSketch** because both systems use hand-related visual information and gesture characteristics for human-computer interaction. However, AirSketch uses **MediaPipe hand landmarks from a conventional camera** and relies mainly on fingertip geometry rather than depth maps and deep multimodal sequence models. The paper nevertheless provides useful insights into improving gesture recognition through **temporal information, multimodal features, preprocessing, and robust evaluation**, which could be considered for future improvements to AirSketch.

[6] Hand Gesture Recognition using Image Processing and Feature Extraction Techniques, Ashish Sharma, Anmol Mittal, Savitoj Singh, and Vasudev Awatramani, 2020

Sharma et al. proposed a hand gesture recognition system for identifying and classifying **American Sign Language (ASL) gestures** from images captured under different environmental conditions. The study focuses on improving recognition by applying image preprocessing and feature extraction techniques before classification. The authors compare **Histogram of Oriented Gradients (HOG), Principal Component Analysis (PCA), and Local Binary Patterns (LBP)** with their proposed approach based on **Canny edge detection, ORB feature extraction, and Bag of Features**. The dataset contains **87,000 images representing 29 classes**, including the 26 alphabets, “nothing,” space, and delete gestures.

In the proposed method, the input RGB image is first converted into grayscale and **Canny edge detection** is applied to reduce background noise and retain important hand edges. ORB is then used to detect and extract visual features from the hand. The extracted descriptors are grouped using **K-means clustering** to generate a Bag-of-Words representation containing 150 visual codewords. These features are subsequently classified using Random Forest, Naïve Bayes, SVM, Logistic Regression, KNN, and Multilayer Perceptron. The results show that the proposed ORB-based approach achieves **95.81% accuracy with KNN and 96.96% with MLP**, demonstrating that compact feature representations can provide effective gesture classification.

This study is relevant to **AirSketch** because both systems use **computer vision techniques to interpret hand information for human-computer interaction**. The paper demonstrates the importance of preprocessing, feature extraction, and classification in improving recognition performance, while AirSketch uses **MediaPipe hand landmarks and fingertip coordinates** for real-time interaction rather than traditional image descriptors. The authors also identify an important limitation: their system was tested only on **static gesture images**, with future work

proposed for dynamic real-time gesture recognition. This distinction is relevant to AirSketch, which operates continuously on webcam frames and uses hand movement and pinch-based interaction

[7] Computer Vision-Based Hand Gesture Recognition for Human-Robot Interaction: A Review, Jing Qi, Li Ma, Zhenchao Cui, and Yushu Yu, 2024.8 Research Gap

Qi et al. presented a comprehensive review of **computer vision-based hand gesture recognition for human-robot interaction (HRI)**. The paper examines how hand gestures can provide natural and intuitive communication between humans and intelligent systems, particularly robots. The authors classify gesture recognition approaches into **sensor-based and vision-based methods**, discussing data gloves, EMG, Wi-Fi/radar, monocular cameras, and RGB-D cameras. For vision-based recognition, the review organizes the complete recognition process into four major stages: **data acquisition, gesture detection and segmentation, feature extraction, and gesture classification**.

The review discusses numerous techniques used at each stage, including image grayscaling, smoothing, edge detection, morphological processing, thresholding, skin-color segmentation, contour-based segmentation, and motion tracking. For feature extraction, both global and local features such as **HOG, LBP, SIFT, SURF, PCA, and LDA** are examined. The paper also compares traditional classification techniques such as template matching, DTW, HMM, SVM, and KNN with deep learning approaches. The authors note that deep learning can automatically extract gesture features and provide greater robustness, although it requires larger datasets and higher computational resources.

The paper identifies several challenges that remain important for real-world gesture recognition, including **complex backgrounds, lighting variations, hand occlusion, camera distance, hand anatomy, training-data limitations, and real-time processing requirements**. It highlights the need for more diverse datasets and improved recognition speed and reliability. The review concludes that RGB-D and vision-based approaches have strong potential for dynamic and static

gesture recognition in robotic applications, while future systems are expected to become more accurate, real-time, reliable, and personalized. This work is relevant to **AirSketch** because it provides a broader foundation for understanding the computer-vision pipeline behind AirSketch's camera-based hand tracking and highlights areas such as segmentation, feature extraction, real-time performance, and robustness that are important for gesture-controlled interfaces.

[8] Hand Gesture and Voice-Controlled Mouse for Physically Challenged Using Computer Vision, Aarti Morajkar, Atheena Mariyam James, Minoli Bagwe, Aleena Sara James, and Aruna Pavate, 2023

Morajkar et al. proposed a **hand gesture and voice-controlled virtual mouse** designed to provide a hands-free alternative to conventional mouse interaction. The system uses computer vision and deep learning to recognize hand gestures and map them to different computer operations. The authors use an **EfficientNetB4-based CNN trained on a custom dataset** for gesture recognition, while MediaPipe can be incorporated for hand tracking. The main objective is to provide a cost-effective and intuitive HCI system without requiring additional specialized hardware such as sensors or controllers.

The proposed system supports several mouse and computer operations, including **cursor movement, left click, right click, double click, scrolling, drag and drop, multiple-item selection, volume control, and brightness control**. A voice assistant is also integrated to provide additional hands-free commands such as file navigation, Google search, copy and paste, and system control. The authors tested the system under different conditions, including varying camera distances, lighting conditions, backgrounds, and the use of left, right, and both hands. The results showed high gesture-classification performance, although the authors observed a delay of a few seconds for mouse actions.

This study is highly relevant to **AirSketch** because both systems use a **camera-based, touchless interface to convert hand gestures into computer actions**. However, the purpose and

implementation differ: the proposed virtual mouse focuses on replacing conventional mouse operations using a trained deep-learning gesture classifier, whereas AirSketch uses **MediaPipe hand landmarks and fingertip-distance-based pinch detection** to enable virtual drawing and tool selection. The paper also highlights practical challenges such as lighting and skin-color variations, which are important considerations for camera-based gesture systems. Its future work includes expanding interaction functions and exploring eye-based control for users unable to use their hands

[9] MediaPipe-LSTM-Enhanced Framework for Real-Time Dynamic Sign Language Recognition in Inclusive Communication Systems, N. Anithadevi, SatheeshKumar Palanisamy, S. Saranya Rubini, and Sachin Shrestha, 2025

Anithadevi et al. proposed a **MediaPipe-LSTM-based framework for real-time dynamic sign language recognition**, focusing on continuous gesture sequences rather than isolated static hand gestures. The system combines **MediaPipe Holistic** for extracting hand, facial, and body landmarks with an enhanced stacked **Long Short-Term Memory (LSTM)** network for learning temporal relationships between consecutive gestures. A dedicated dataset containing dynamic Indian Sign Language gestures was developed, while preprocessing includes frame extraction, resizing, normalization, data augmentation, and landmark extraction. The system further incorporates a transformer-based paraphrasing module that converts recognized gesture sequences into semantically meaningful text and search queries.

The proposed model uses **three LSTM layers followed by dense layers** for sequential gesture classification. MediaPipe provides structured landmark features, allowing the model to focus on meaningful hand, face, and posture information instead of processing raw video alone. The authors report **97% accuracy, 98.17% precision, and 94.88% F1-score**, with the comparative evaluation indicating approximately **25 ms per frame** processing time. Their ablation study further demonstrated the contribution of each component: removing MediaPipe reduced accuracy by **7.8%**,

removing attention reduced it by **4.5%**, and replacing LSTM with a CNN resulted in an **11.4% decrease**, highlighting the importance of temporal sequence modeling.

This work is highly relevant to **AirSketch** because both systems use **webcam input, MediaPipe landmarks, and real-time hand movement analysis**. However, their objectives differ: the proposed system performs continuous sign-language recognition and converts gesture sequences into meaningful language, whereas AirSketch uses hand landmarks primarily for **drawing interaction and gesture-based tool control**. The paper provides useful direction for future AirSketch improvements because it demonstrates how temporal models such as LSTM can capture movement patterns across multiple frames instead of relying only on individual-frame fingertip distances. It also emphasizes the importance of robust preprocessing, balanced datasets, and evaluation using precision, recall, and F1-score.

[10] Towards Understanding the Faults of JavaScript-Based Deep Learning Systems, Lili Quan, Qianyu Guo, Xiaofei Xie, Sen Chen, Xiaohong Li, and Yang Liu, 2022

Quan et al. presented an empirical study on the **faults and quality issues of JavaScript-based deep learning systems**, with a particular focus on TensorFlow.js. The authors analyzed **700 real-world faults** collected from 72 GitHub repositories, including TensorFlow.js, third-party deep learning libraries, and web-based deep learning applications. The study categorizes these faults according to their symptoms, root causes, and fixing methods. The authors identified **26 fault symptoms, 17 root causes, and 16 fix patterns**, while also studying faults across different stages of the deep learning development lifecycle.

The study found that **crashes were the most common fault**, accounting for 318 of the 700 identified faults, followed by build and initialization failures and poor-performance problems. Important root causes included incorrect code logic, dependency errors, API misuse, data or model inaccessibility, device incompatibility, browser incompatibility, and limitations of JavaScript-specific backends such as WebGL and Wasm. The authors also examined common

solutions, including changing software versions, modifying dependencies, changing execution environments or backends, correcting model paths, and modifying API usage. These findings demonstrate that JavaScript-based deep learning systems can face problems not only in their models but also in their supporting software and execution environments.

This work is relevant to **AirSketch** mainly from a software reliability and implementation perspective. AirSketch depends on computer-vision libraries and continuous real-time processing, meaning that issues such as incorrect API usage, dependency problems, processing delays, and runtime errors can affect its performance. Unlike the gesture-recognition papers discussed earlier, this study does not propose a new gesture-recognition method. Instead, it provides useful guidance for **testing, debugging, performance evaluation, and reliable implementation** of AI-based interactive systems. Its findings can therefore support future improvements to AirSketch by encouraging evaluation of the complete system rather than focusing only on gesture-recognition accuracy.

[11] Painting with Hand Gestures using MediaPipe, R. Vasavi, Nenavath Rahul, A. Snigdha, K. Jeffery Moses, and S. Vishal Simha, 2022

Vasavi et al. proposed a **real-time painting application controlled through hand gestures**, allowing users to draw on a virtual canvas without using a mouse, keyboard, or other physical input device. The system uses a webcam to capture hand movements and combines **OpenCV and MediaPipe** for hand detection and tracking. MediaPipe's palm detection and hand landmark models are used to identify the hand and locate the **index fingertip**, which acts as the primary input for drawing. The authors describe the system as a contactless human-computer interaction approach and highlight its potential applications in areas such as virtual environments, sign-language interaction, industrial automation, and rehabilitation.

The application continuously processes webcam frames and determines the user's hand and finger positions using MediaPipe. When the **index finger is raised**, the user can draw on the canvas

according to the movement of the fingertip. Other hand configurations are used for additional operations, such as moving around the screen, selecting a colour, and clearing the canvas. The paper emphasizes MediaPipe's low-latency processing, modular structure, and ability to provide **21 hand landmark coordinates**, while OpenCV is responsible for image and video processing. The results demonstrate that the system can detect the live hand, track the index fingertip, and display freehand drawings and writing on the virtual canvas in real time.

This work is highly relevant to **AirSketch** because both systems use **webcam-based hand tracking, MediaPipe landmarks, fingertip movement, and a virtual drawing canvas** for contactless interaction. However, AirSketch extends this basic approach by using **pinch-based interaction** and providing multiple drawing tools such as brush, eraser, lines, circles, and rectangles. The paper therefore provides direct support for AirSketch's choice of MediaPipe and OpenCV as the core computer-vision technologies. It also demonstrates that fingertip tracking can provide responsive real-time drawing, while the paper notes that variations in hand position, orientation, hand size, and lighting remain challenges for vision-based gesture systems.

[12] Dynamic LIBRAS Gesture Recognition via CNN over Spatiotemporal Matrix Representation, Jasmine Moreira, 2026

Moreira proposed a **dynamic hand gesture recognition system** that combines the **MediaPipe Hand Landmarker** with a convolutional neural network (CNN) to recognize LIBRAS gestures for home automation control. MediaPipe extracts 21 hand keypoints from each video frame, which are then organized into a **90×21 spatiotemporal matrix** representing the x, y, and z coordinates across 30 frames. This matrix is converted into a grayscale image and classified using a lightweight CNN. The system covers **11 static and dynamic gesture classes** and uses a sliding-window approach for continuous real-time recognition without requiring recurrent neural networks.

The proposed CNN contains three convolutional layers and two dense layers, with the selected configuration containing approximately **25,000 parameters**. During real-time inference, each

captured frame is triplicated in a buffer so that overlapping windows can represent continuous movements. Predictions are accepted only when the confidence exceeds 98%. The system achieved **95% accuracy under low-light conditions and 92% under normal lighting**. However, the experiments were performed using a single user, and the authors note that the results may not generalize to users with different hand anatomy, gesture speed, or expression style.

This work is relevant to **AirSketch** because both systems use **MediaPipe hand landmarks for real-time gesture-based interaction**. The main difference is that AirSketch currently uses fingertip geometry and pinch detection to control drawing, whereas this paper processes sequences of landmarks with a CNN to recognize complete static and dynamic gestures. Its use of temporal information and overlapping windows could be useful for future AirSketch improvements involving more complex gestures or commands. The paper also highlights the importance of testing across different users and lighting conditions, which are relevant considerations for improving the robustness of AirSketch.

[13] AirSketch: Generative Motion to Sketch, Hui Xian Grace Lim, Xuanming Cui, Yogesh S. Rawat, and Ser-Nam Lim, 2024

Lim et al. introduced **AirSketch**, a marker-less air-drawing approach that generates clean and visually coherent sketches directly from hand movements captured using a standard camera. The authors address limitations of conventional AR/VR drawing systems, which may require headsets, controllers, or digital markers. Their approach first uses a hand-tracking algorithm to obtain a noisy representation of the user's hand motion and then applies a **controllable image diffusion model** to transform this noisy input into a clean sketch. The authors created both synthetic and real air-drawing datasets to evaluate the proposed approach.

The proposed method uses **ControlNet with Stable Diffusion XL (SDXL)** and an augmentation-based self-supervised training procedure. During training, clean sketches are deliberately modified using different types of noise, including hand jitters, stroke distortions,

incorrect positioning, and false strokes, allowing the model to learn how to recover the intended sketch. The real air-drawing dataset contains videos recorded at **1280×720 resolution and 30 FPS**. With augmentation-based training, the model improved the similarity between generated and ground-truth sketches; for the real dataset, SDXL achieved an SSIM of **0.64**, compared with 0.55 for the original tracking input, while Chamfer Distance decreased from 32.36 to 25.46.

This work is relevant to **AirSketch** because it demonstrates how noisy hand-tracking data can be converted into cleaner and more aesthetically consistent drawings. Both systems rely on **camera-based hand tracking**, with the paper specifically noting that MediaPipe can provide 21 hand landmarks from RGB images or video. However, the proposed system differs significantly from the AirSketch project because it focuses on **generative sketch refinement using diffusion models**, whereas AirSketch provides direct real-time drawing and interaction through gestures and drawing tools. The paper's discussion of tracking errors, hand jitter, unintended strokes, and false strokes is particularly relevant to improving the robustness and visual quality of AirSketch in future work.

[14] Virtual Write and Clear Application: A Gesture-based Drawing Tool using Mediapipe and OpenCV, Samruddhi Barade, Payal Kadam, Pradnesh Vedpathak, and N.S. Bahirame, 2025

Barade et al. presented a **gesture-based virtual drawing application** that allows users to draw without traditional input devices such as a mouse, stylus, or touchscreen. The system uses **Python, OpenCV, and MediaPipe** to track hand movements through a webcam and map them to actions on a virtual canvas. MediaPipe provides 21 hand landmarks for identifying gestures, while the user's index-fingertip movement is used to draw lines in real time. The application also includes gestures for clearing the canvas and changing colours, providing a contactless human-computer interaction system.

The system follows a modular architecture consisting of **video capture, hand tracking, gesture recognition, drawing interface, and user interaction** modules. Specific gestures are mapped to

functions such as colour selection and erasing; for example, raising the thumb and index finger together can trigger a colour change. The authors report that the application maintained an average latency of **50 ms**, while hand-tracking accuracy was reported to be above **95%**. The results demonstrate real-time hand tracking, drawing, colour selection, and canvas clearing through gestures.

This work is highly relevant to **AirSketch** because both systems use **MediaPipe, OpenCV, webcam-based hand tracking, and gesture-controlled virtual drawing**. However, AirSketch extends this concept with pinch-based interaction and a wider range of drawing tools, including geometric shapes, brush and eraser controls. The paper's modular architecture is also closely related to AirSketch's separation of hand tracking, gesture detection, and drawing functionality. Its reported latency and accuracy provide useful reference points when evaluating the responsiveness and reliability of AirSketch. The authors also identify future improvements such as advanced gesture recognition and multi-hand support, which could be considered for further development

[15] Virtual Hand Gesture Painting, Satyam V. Tiwari, Deep A. Vartak, Vivek R. Verma, Anasali I. Sayyed, and Ms. Soniya Khatu, 2022

Tiwari et al. proposed a **virtual hand gesture painting system** that enables users to draw and write on a digital canvas without relying on physical input devices. The system uses a webcam along with **OpenCV and MediaPipe** to detect and track hand movements in real time and translate them into drawing actions. MediaPipe is used for hand tracking and landmark detection, while the user's finger movements are mapped to the virtual canvas. The system supports drawing, colour selection, erasing, and shape creation, providing a contactless approach to digital painting.

The proposed system detects the user's hand through the camera and provides different interaction modes based on the detected fingers. When only the **index finger** is raised, the user enters drawing mode, while the index and middle fingers together are used to select options such as the brush, shapes, and eraser. The system also allows users to select different colours from the interface. The

implementation section demonstrates real-time hand landmark detection and drawing by moving the index finger in front of the camera. The authors further describe connecting the system to a mobile device so that the drawing output can also be displayed on the phone.

This work is highly relevant to **AirSketch** because it uses a very similar combination of **OpenCV**, **MediaPipe**, **fingertip tracking**, **gesture-based tool selection**, and **virtual drawing**. It is particularly close to AirSketch in its use of separate gestures for drawing, selecting tools, shapes, and erasing. However, AirSketch uses **pinch-based interaction** and includes additional drawing and evaluation features. The paper also identifies challenges such as variations in hand size, position, orientation, and lighting conditions, which are important considerations for improving AirSketch's robustness.

III. EXISTING AND PROPOSED SYSTEM

3.1 Existing Systems

3.1.1 Microsoft Kinect

Microsoft Kinect represents a class of gesture-interaction systems that combine RGB imaging with depth and infrared sensing. Such systems can observe larger body movements and provide richer spatial information than a conventional RGB webcam. They have been used for gaming, interactive installations, rehabilitation research, and other motion-controlled applications.

For the AirSketch use case, the important limitation is the dependency on dedicated sensing hardware. The need for a specialised sensor increases cost and reduces portability compared with a system that uses a webcam already available on a laptop or desktop computer.

3.1.2 Leap Motion Controller

The Leap Motion Controller uses optical sensing to track hands and fingers at high spatial detail. It is designed specifically for hand interaction and has been used in virtual reality, 3D modelling,

simulation, and interactive systems. Its dedicated sensing capability can provide precise finger information, but it introduces an additional hardware dependency and requires suitable placement and calibration.

3.1.3 Conventional Digital Drawing

Mouse- and stylus-based drawing systems provide mature and precise interaction. However, they require physical contact and are not designed for touchless use. Touchscreens remove the mouse requirement but still depend on physical contact with a display surface. AirSketch explores a different interaction model in which the user's fingertip acts as the pointer and pinch acts as the activation mechanism.

Limitations of Existing Approaches

- Specialized sensors can increase cost and reduce portability.
- Depth and multimodal systems may require additional computation and calibration.
- Marker- or colour-based approaches can require an extra physical object or controlled visual conditions.
- Learned dynamic gesture systems require datasets, training, validation, and model maintenance.
- Simple drawing systems may not separate pointer movement from intentional drawing clearly.
- Real-time camera systems remain sensitive to lighting, background clutter, motion blur, and hand occlusion.

3.2 Proposed AirSketch System

The proposed AirSketch system uses a conventional RGB webcam and a Python-based real-time processing loop. The webcam supplies frames at the requested 1280×720 working resolution. OpenCV horizontally flips each frame before the frame is passed to the virtual painter. MediaPipe

Hands estimates 21 hand landmarks. The index and middle fingertip coordinates are extracted, and their Euclidean distance determines the pinch state.

The current implementation supports eight tools: brush, eraser, rectangle, circle, line, filled rectangle, filled circle, and clear canvas. Twelve colours are available through the toolbar. Brush thickness is selected from ten values: 5, 10, 15, 20, 25, 30, 35, 40, 45, and 50 pixels. A temporary canvas is used for shape previews so that the persistent canvas is not modified until the user releases the pinch. Gesture-based interaction eliminates the need for physical input devices during drawing operations. The system maintains the selected tool, colour, and brush thickness throughout the drawing session.

3.3 Feature Set

Table 3.3.1 in the Feature Set subsection summarizes the key features of the proposed AirSketch system and specifies how each feature is implemented. Its purpose is to provide a concise overview of the system's interaction mechanisms, drawing capabilities, configuration options, and evaluation procedures. The table establishes the implementation details of each feature, making it easier to understand how hand tracking, gesture recognition, drawing operations, and user-interface controls work together within the proposed system.

Table 3.3.1 Feature Set

Feature	Implementation Behaviour
Hand tracking	MediaPipe Hands; one detected hand; 21 landmarks
Virtual pointer	Index fingertip landmark 8
Activation	Pinch when distance between landmarks 8 and 12 is below 70 pixels
Smoothing	Five-sample arithmetic moving average
Freehand brush	Connect successive smoothed fingertip points
Eraser	Draw black trajectory using current brush thickness
Shapes	Line, rectangle, circle, filled rectangle, filled circle

Colour selection	Fingertip selection from twelve toolbar colour circles
Brush size	Ten levels from 5 to 50 pixels
Canvas clear	Reinitialize persistent 720×1280 canvas
Toolbar debounce	0.3 seconds between UI selections
Evaluation	Frame-level labelled pinch evaluation, CSV and text report

IV. PROBLEM DEFINITION, OBJECTIVES AND PROPOSED METHODOLOGY

4.1 Problem Definition

Conventional digital drawing interfaces depend on physical input devices. Although these devices are effective, they require contact and can be inconvenient in situations where touchless interaction is preferred. Existing gesture systems can also introduce dedicated hardware, complex models, or limited application functionality. The problem addressed by AirSketch is therefore to design and implement a real-time virtual drawing interface that can translate natural hand movement from a standard RGB webcam into controlled drawing actions while maintaining a clear distinction between pointer movement and intentional activation.

4.2 Objectives

1. To develop a real-time gesture-controlled virtual drawing application using a standard RGB webcam.
2. To detect and track hand landmarks using MediaPipe.
3. To use a deterministic and interpretable gesture rule for drawing activation.
4. To reduce fingertip jitter through lightweight smoothing.
5. To provide freehand drawing and geometric drawing primitives.

6. To provide colour selection, brush-size adjustment, erasing, and canvas clearing.
7. To evaluate pinch detection using labelled samples and standard classification metrics.
8. To document limitations and identify technically plausible future enhancements.

4.3 Hardware Requirements

The current implementation requires a computer capable of running Python, a working webcam, and sufficient CPU resources for real-time MediaPipe processing. No depth camera, wearable sensor, or graphics-processing unit is specified by the repository. The actual experience can still vary with camera quality, lighting, processor performance, and available memory.

4.4 Software Requirements

Table 4.4.1 presents the software requirements used for developing and executing the proposed AirSketch system. The table identifies the major software packages and libraries, their corresponding versions used in the repository, and their specific roles within the application. These components collectively support the core functionality of the system, including application execution, real-time hand landmark detection, camera input, image processing, canvas rendering, numerical operations, and evaluation logging. By consolidating these dependencies in one place, the table provides a clear overview of the software environment required for implementation and helps establish the technical configuration necessary to reproduce and operate the proposed system effectively.

Software / Package	Version in Repository	Purpose
Python	Project runtime	Application execution
MediaPipe	0.10.21	Hand landmark detection and tracking

OpenCV	4.10.0.84	Camera input, rendering, image operations, display
NumPy	1.26.4	Canvas arrays and numerical operations
CSV / datetime	Python standard library	Evaluation logging and timestamps

Table 4.4.1 Software Requirements

4.5 Feasibility Analysis

4.5.1 Technical Feasibility

The project is technically feasible because its main components are available as established Python packages and the interaction logic is deterministic. The system uses a conventional RGB webcam for image acquisition and does not depend on specialized sensing equipment. OpenCV provides the required image-processing and camera-handling functionality, while MediaPipe Hands provides real-time hand landmark detection. The current repository separates camera acquisition, hand tracking, drawing, user interface, and evaluation into individual modules, making the system easier to maintain, test, and modify.

4.5.2 Economic Feasibility

The system is economically feasible because it does not require dedicated depth cameras, motion sensors, graphics tablets, or other specialized hardware. A conventional RGB webcam is sufficient for capturing the user's hand movements, making the system suitable for implementation on commonly available computers and laptops. This reduces the initial hardware investment compared with gesture-based systems that depend on specialized sensing devices.

4.5.3 Operational Feasibility

The system is operationally feasible because its interaction model is simple and can be performed using natural hand movements. The user moves the fingertip to position the virtual cursor and uses a pinch gesture between the index and middle fingers to activate drawing or tool operations. The toolbar provides direct access to drawing tools, colours, brush thickness, and canvas controls, reducing the need to remember multiple complex gestures.

4.6 Methodology

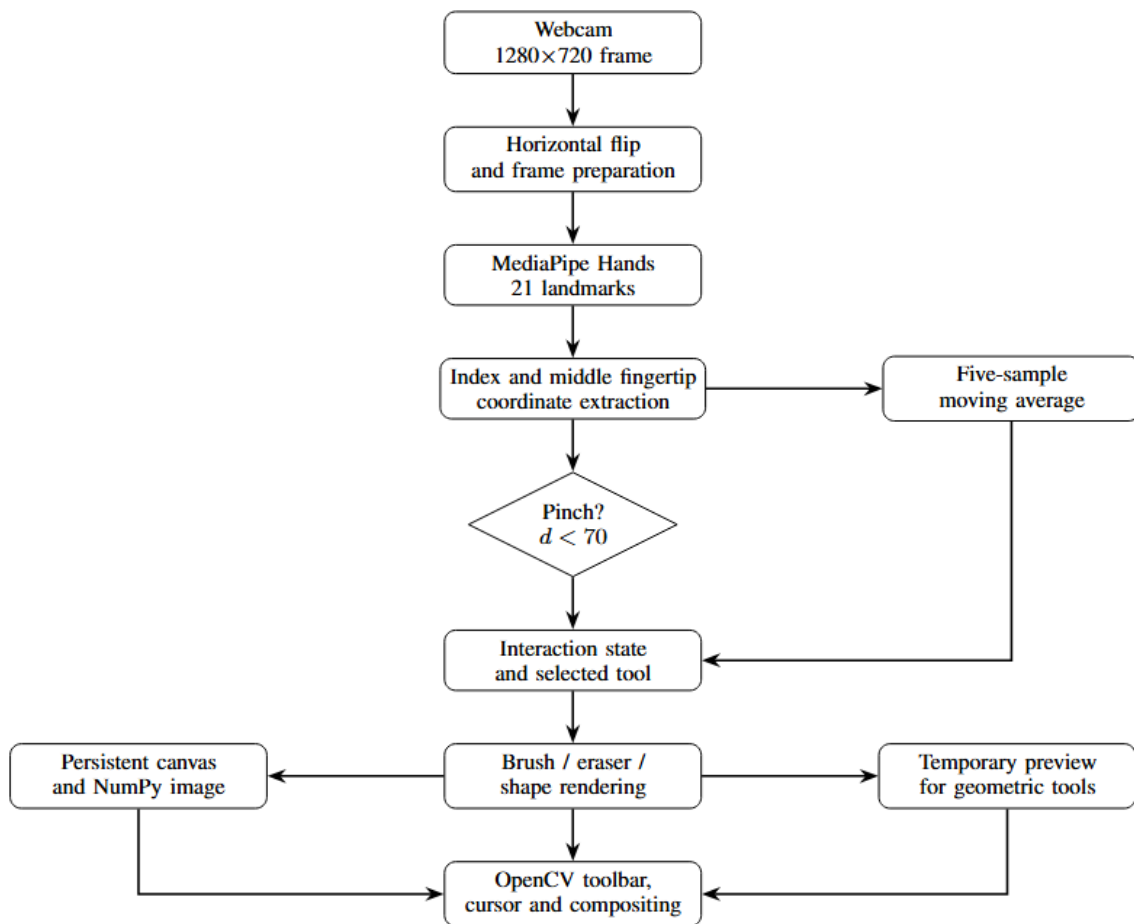


Figure 4.6.1 Methodology

Figure 4.6.1: Architecture and Processing Pipeline illustrates the overall processing workflow of the proposed AirSketch system, from camera input to final visualization. The process begins with a **1280×720 webcam frame**, which undergoes horizontal flipping and frame preparation. **MediaPipe Hands** then detects a single hand and extracts its 21 landmarks. The index and middle fingertip

coordinates are obtained and passed through a **five-sample moving average** to reduce pointer jitter. The system determines whether a pinch gesture is active by checking whether the distance between the fingertips is less than 70 pixels. Geometric tools additionally generate a temporary preview until the interaction is completed. The resulting drawing is maintained on a persistent NumPy canvas, while the OpenCV toolbar, cursor, and compositing processes combine the canvas and interface elements to produce the final displayed output.

V. SYSTEM DESIGN

5.1 Block Diagram Of Project

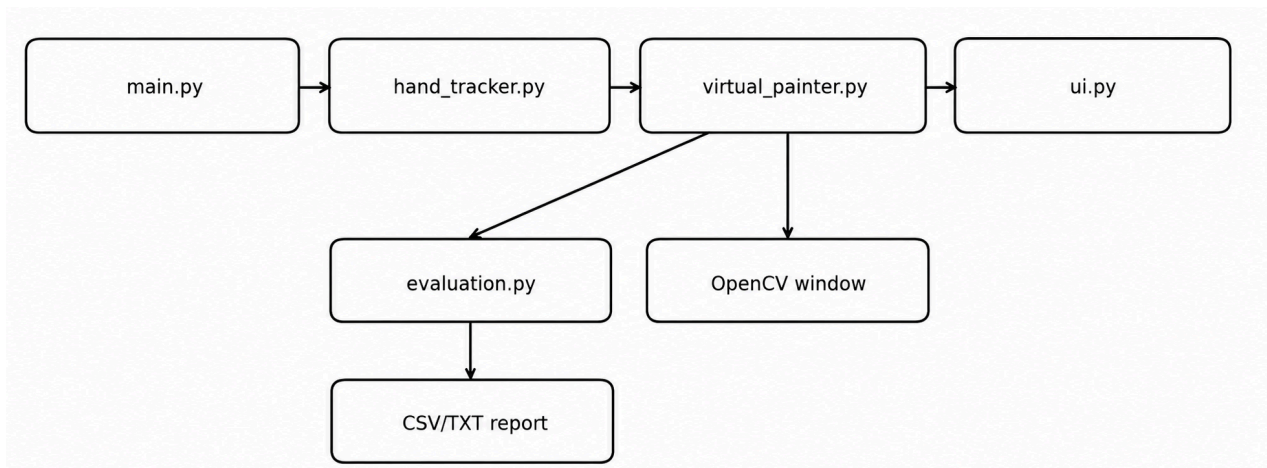


Fig 5.1.1 Block Diagram of Project

In the fig 5.1.1, the main program creates a VirtualPainter object and enters a continuous loop. Each captured frame is flipped and passed to process_frame. VirtualPainter calls HandTracker.process, updates interaction state, performs drawing or UI operations, composites the result, and asks UI to render the toolbar. When evaluation mode is enabled, VirtualPainter passes the predicted pinch state and fingertip distance to GestureEvaluator. The evaluator periodically records samples and calculates the confusion matrix and derived metrics.

The processing pipeline begins with the default webcam. The main module requests a 1280×720 capture size and horizontally flips each frame. The hand tracker converts the BGR frame to RGB before calling MediaPipe Hands. If no hand is detected, the tracker returns None and the persistent

canvas remains unchanged. If a hand is detected, landmarks 8 and 12 are converted from normalized coordinates to pixel coordinates.

The landmark processor calculates the Euclidean distance between the two fingertip points. A value below 70 pixels sets the pinch state to true. The index fingertip is passed through a five-sample moving average before it is used as the interaction point. The virtual painter then decides whether the pointer is in the toolbar or the drawing region. In the toolbar, the UI module handles selection. In the drawing region, the pinch state controls drawing and shape operations.

5.2 Module Description

Module	Primary Responsibility	Important Data / State
main.py	Camera, application loop, keyboard evaluation controls	capture object, painter, evaluation mode
hand_tracker.py	MediaPipe initialization, landmarks, pinch, smoothing	21 landmarks, fingertip history, pinch state
virtual_painter.py	Canvas, drawing state, tools, compositing	canvas, temp_canvas, tool, colour, thickness
ui.py	Toolbar drawing and pointer-based controls	colour positions, tool bounds, size controls
evaluation.py	Ground-truth collection and metrics	TP, TN, FP, FN, samples, CSV log

Table 5.2.1 Module description

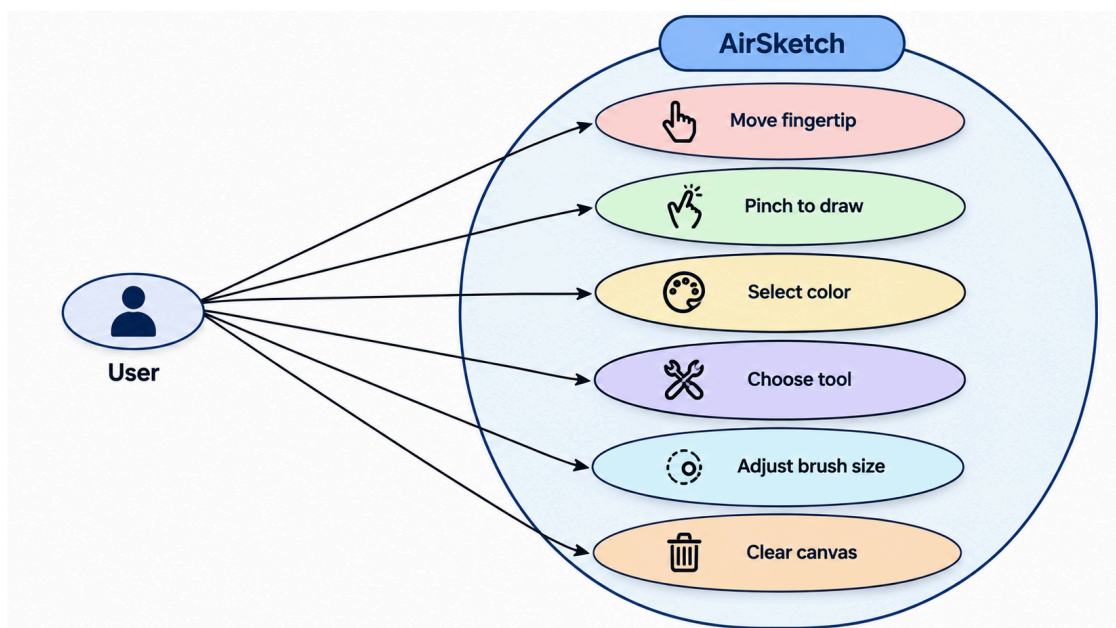
Table 5.2.1 presents the modular structure of the proposed AirSketch system and summarizes the primary responsibility and important data or state associated with each software module. The table provides an overview of how the system is divided into separate functional components, improving clarity and maintainability. The `main.py` module manages camera input and the application loop, while `hand_tracker.py` handles hand detection, landmark processing, pinch recognition, and smoothing. The `virtual_painter.py` manages the canvas and drawing operations, whereas `ui.py` controls toolbar interactions. Finally, `evaluation.py` handles ground-truth collection, performance metrics, and CSV-based evaluation logging.

5.3 Data Flow

The primary data object is a video frame represented as a NumPy array. MediaPipe receives an RGB representation and returns hand landmarks. The relevant landmark coordinates are converted into integer pixel coordinates. The smoothed fingertip position becomes the control point for UI selection or drawing. The persistent canvas is another NumPy array of shape $720 \times 1280 \times 3$. A temporary canvas is used only while previewing geometric shapes.

5.4 User Interaction Model

In the fig 5.4.1 the interaction model can be understood as a small state machine. In the pointer state, the user can move the fingertip without creating a stroke. When the pinch becomes active in the drawing region, a drawing interaction starts. For brush and eraser, subsequent positions extend the stroke. For geometric tools, subsequent positions update a temporary preview. When the pinch is released, the selected shape is committed to the persistent canvas. The toolbar forms a separate interaction region and takes precedence over drawing when the pointer is above the configured boundary.



Figure

5.4.1: Primary AirSketch User Interactions.

5.5 User Interface Design

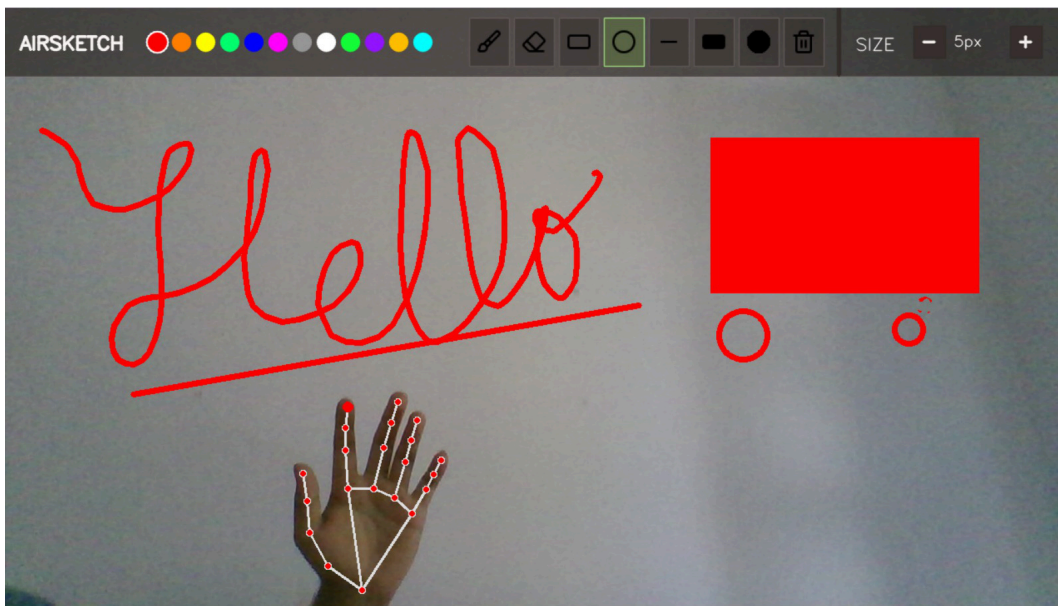


Figure 5.5.1: Representative AirSketch drawing and shape-creation interface from the research paper.

The current UI is drawn directly onto the OpenCV frame as displayed in the fig 5.5.1, allowing the controls and drawing area to be displayed within the same application window. The toolbar height is 82 pixels and occupies the upper portion of the interface. It displays the AIRSKETCH title, twelve colour circles, eight tool buttons, and brush-size controls. Tool icons are loaded from PNG assets to provide clear visual representations of the available operations. The selected tool is visually distinguished from the other buttons, providing immediate feedback about the current drawing mode.

The UI interaction handler applies a 0.3-second debounce interval. This is intended to prevent repeated selection when the fingertip remains over a control across several frames. Importantly, the debounce is applied to toolbar selection rather than to the continuous drawing trajectory, ensuring that drawing remains responsive and smooth during hand movement. The combination of visual feedback, simple spatial selection, and debounce handling provides a consistent interaction mechanism for the virtual canvas.

VI. SYSTEM IMPLEMENTATION

6.1 System Implementation

6.1.1 Development Environment

The current repository implementation is a Python desktop application. The research paper specifies MediaPipe 0.10.21, OpenCV 4.10.0.84, and NumPy 1.26.4. OpenCV handles camera acquisition, image conversion, drawing primitives, alpha compositing, and display. MediaPipe provides the hand landmark representation. NumPy provides the array representation used for the persistent and temporary canvases

6.1.2 Main Application Module

The main module creates the camera object, configures the requested resolution, creates a VirtualPainter instance, and enters the frame-processing loop. It also provides keyboard controls for the evaluation experiment. Pressing E toggles evaluation mode, 1 labels the current ground truth as pinch, 0 labels it as no pinch, R resets counters, S saves the evaluation report, and Q exits the application.

```
def main():  
    cap = cv2.VideoCapture(0)  
    cap.set(3, 1280)  
    cap.set(4, 720)  
    painter = VirtualPainter()  
    while True:  
        ret, frame = cap.read()  
        frame = cv2.flip(frame, 1)  
        final_frame = painter.process_frame(frame)  
        cv2.imshow(painter.window_name, final_frame)
```

Code fragment: simplified application loop based on the repository.

6.1.3 Hand Tracker Module

HandTracker encapsulates MediaPipe initialization and landmark processing. It converts BGR frames to RGB, invokes MediaPipe, draws the detected hand connections for visual feedback, extracts landmarks 8 and 12, computes their distance, determines pinch state, and smooths the index fingertip. When no hand is detected, the module returns None.

6.1.4 Virtual Painter Module

VirtualPainter is the central application object. It initializes a $720 \times 1280 \times 3$ persistent canvas and temporary canvas, defines twelve colours, defines the ten brush-size levels, lists the eight tools, and creates the hand tracker, UI, and evaluator. Its process_frame method orchestrates all stages from hand data acquisition through drawing and final display.

The brush method joins the previous point and current point with cv2.line. The eraser uses the same trajectory but renders in black. Geometric tools are drawn on temp_canvas during interaction. On release, the final primitive is drawn onto the persistent canvas. The class then performs mask-based compositing and asks the UI object to render the toolbar.

6.1.5 User Interface Module

The UI module uses OpenCV drawing operations to create the toolbar. It loads eight PNG tool icons and supports alpha compositing. The toolbar contains twelve colour choices, eight tools, and brush-size controls. Selected tools are visually highlighted. The UI module uses pointer coordinates to identify which control has been selected and applies a 0.3-second debounce interval. The toolbar is rendered on every processed frame so that the available controls remain visible during interaction.

6.1.6 Evaluation Module

GestureEvaluator provides a frame-level evaluation mechanism. The evaluator maintains TP, TN, FP, and FN counters. During evaluation mode, a user supplies ground truth by pressing 1 for pinch

or 0 for no pinch. The evaluator samples every third frame by default. For each accepted sample, it records a timestamp, actual label, predicted label, and fingertip distance in `evaluation_results.csv`.

During evaluation mode, the user supplies the ground truth by pressing `1` when the observed gesture represents a pinch and `0` when it represents a no-pinch state. To reduce redundant samples while maintaining sufficient evaluation data, the evaluator samples every third frame by default. For each accepted sample, the system records the timestamp, actual label, predicted label, and fingertip distance in `evaluation_results.csv`. This recorded data can subsequently be used to verify the calculated performance metrics and analyse individual prediction errors.

```
accuracy = (tp + tn) / total
precision = tp / (tp + fp) if (tp + fp) else 0.0
recall = tp / (tp + fn) if (tp + fn) else 0.0
f1 = 2 * precision * recall / (precision + recall)
```

Code fragment: core evaluation metric calculations.

6.1.7 Configuration and Dependencies

The repository requirements file pins three packages: `mediapipe==0.10.21`, `opencv-python==4.10.0.84`, and `numpy==1.26.4`. Pinning these versions improves reproducibility of the current environment. The standard-library modules `csv`, `os`, `datetime`, `math`, `time`, and `collections` are used by the source files and do not require separate installation.

6.2 ALGORITHMS

6.2.1 Video Acquisition

The main program initializes `OpenCV VideoCapture(0)`, requests a working resolution of `1280×720`, and continuously reads frames. Each valid frame is horizontally flipped before processing.

```
cap = cv2.VideoCapture(0)
cap.set(3, 1280)
```

```
cap.set(4, 720)

ret, frame = cap.read()

frame = cv2.flip(frame, 1)
```

Code fragment: camera initialization and mirror-view preparation.

6.2.2 Hand Landmark Detection

The hand tracker initializes MediaPipe Hands with `max_num_hands=1`, minimum detection confidence 0.75, and minimum tracking confidence 0.70. MediaPipe provides 21 landmarks.

```
self.hands = self.mp_hands.Hands(

    max_num_hands=1,

    min_detection_confidence=0.75,

    min_tracking_confidence=0.7

)
```

Code fragment: MediaPipe Hands configuration in the current repository.

6.2.3 Coordinate Mapping

MediaPipe provides normalized landmark coordinates. The current implementation converts the index and middle fingertip coordinates into the fixed 1280×720 working space. For a normalized point (xn, yn), the current mapping is $x_p = \text{floor}(1280x_n)$ and $y_p = \text{floor}(720y_n)$. This direct mapping simplifies the implementation but creates a dependency on the configured working resolution.

6.2.4 Pinch Detection

The pinch detector uses the Euclidean distance between landmark 8 and landmark 12. If the distance is less than 70 pixels, the system sets `pinch=True`; otherwise `pinch=False`. The rule is deterministic, interpretable, and does not require a trained classifier.

```
distance = math.hypot(x2 - x1, y2 - y1)

pinch = distance < 70
```

Code fragment: deterministic pinch rule.

The threshold is expressed in the 1280×720 coordinate system. Because the threshold is not normalized to hand size, the apparent size of the hand and its distance from the camera can affect the interaction. The research paper identifies normalized thresholding as a future improvement.

6.2.5 Shape Rendering

The shape renderer supports five geometric modes. A rectangle uses the initial and current points as opposite corners. A line uses the same pair as endpoints. A circle uses the initial point as the centre and calculates the radius from the Euclidean distance to the current point. Filled variants use OpenCV's negative thickness parameter. The selected brush thickness is passed to outline primitives.

```
radius = int(math.hypot(x2 - x1, y2 - y1))
cv2.circle(canvas, (x1, y1), radius, color, thickness)
```

Code fragment: circle radius and rendering logic.

6.2.6 Canvas Compositing

The drawing is not displayed as a separate opaque image. The implementation converts the selected drawing surface to grayscale and thresholds it at 5 to create a foreground mask. The inverse mask preserves the camera image in non-drawing regions. The foreground drawing and masked camera image are then added to form the final display. This produces the visual effect of drawing directly over the live camera feed.

6.2.7 Toolbar Interaction and Debouncing

When the smoothed fingertip lies above the toolbar boundary, the UI module takes control. Colour selection tests the distance from the pointer to each colour circle. Tool selection checks whether the pointer lies within each tool button. Brush-size controls increase or decrease the current index within the ten-element thickness list. The clear tool replaces the persistent canvas with a new zero-valued array and resets drawing state.

The UI uses a 0.3-second debounce. This prevents a fingertip remaining over a button from causing repeated selections at the video sampling rate. The separation between toolbar interaction and drawing interaction is therefore achieved by both spatial state and temporal debouncing.

6.2.8 Gesture and Tool Mapping

Table 6.2.8 summarizes the gesture and tool mapping implemented in the proposed AirSketch system, showing how different hand states, pointer positions, and pinch conditions are interpreted by the application. The table establishes the relationship between detected user input and the corresponding system action, covering virtual pointer movement, drawing, erasing, geometric shape creation, and toolbar interaction. It also describes how pinch activation and release control continuous drawing and shape commitment. Additionally, pointer positions within the toolbar are mapped to colour, tool, brush-size, and canvas-clear controls, providing a clear representation of the interaction logic used throughout the system.

Input / Condition	Interpretation	System Action
Hand detected	Index fingertip available	Virtual pointer position
Distance < 70 px	Pinch active	Activate drawing state
Pinch + brush + motion	Continuous brush input	Draw line segments
Pinch + eraser + motion	Continuous eraser input	Draw black erasing path
Pinch + geometric tool	Shape interaction	Update temporary preview
Pinch release + geometric tool	Interaction completed	Commit shape
Pointer in toolbar	UI interaction mode	Interpret position as control selection
Pointer on colour	Colour control hit	Change current colour
Pointer on tool	Tool control hit	Change active tool
Pointer on size control	Size control hit	Increase/decrease thickness
Pointer on clear	Clear control hit	Reset persistent canvas

Table 6.2.8 Gesture and Tool Mapping

VII. EXPERIMENTAL RESULTS AND DISCUSSION

7.1 Experimental Results

7.1.1 Functional Results

The implementation displayed in figure 7.1.1.1 and figure 7.1.1.2 demonstrates the complete interaction pipeline from camera input to drawing output. Representative screenshots show the hand landmarks overlaid on the detected hand, the fingertip used as the interaction point, and drawing content rendered directly over the camera view.

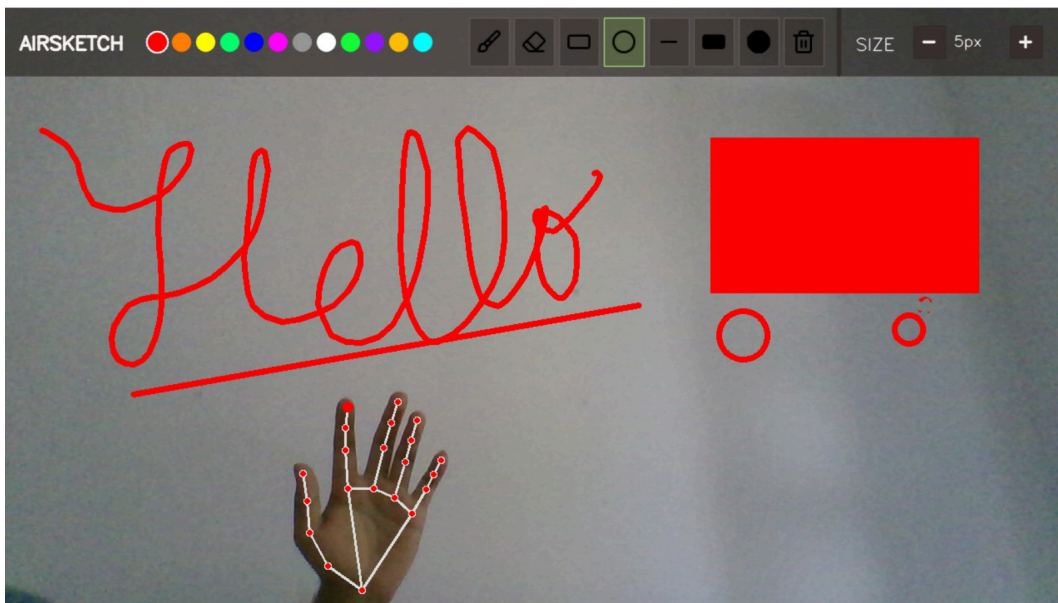


Figure 7.1.1.1: Representative drawing and shape-creation output from AirSketch.

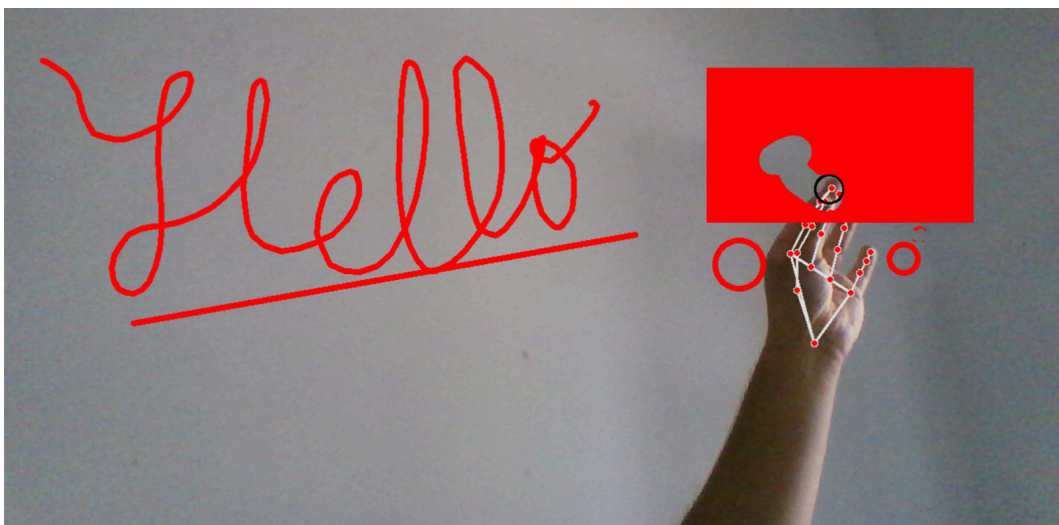


Figure 7.1.1.2: Representative hand-based interaction with previously drawn content.

7.1.2 Drawing and Shape Results

The output demonstrates freehand strokes, a straight line, and geometric shapes within the same interaction session. The brush renders a continuous polyline from successive fingertip positions. The shape tools provide a temporary preview while the user moves the fingertip with an active pinch and commit the selected primitive when the pinch is released. This prevents the persistent canvas from accumulating intermediate shape previews.

The eraser follows the same trajectory mechanism as the brush but uses black pixels and the current brush thickness. The clear tool replaces the persistent canvas with a zero-valued image. These operations demonstrate that the gesture layer is not limited to a single drawing command; it provides a compact control language for multiple canvas operations.

7.1.3 User Interface Results

The toolbar provides a compact set of controls without requiring keyboard-driven tool selection during normal interaction. Twelve colour choices are displayed as circles, eight tools are represented by icons, and brush thickness is controlled with decrement and increment buttons. The selected tool is visually highlighted, providing feedback about the current state.

The pointer can move into the toolbar region without creating a drawing stroke because the toolbar condition takes precedence. The 0.3-second debounce prevents a stationary fingertip from repeatedly triggering the same control. This separation is an important part of the interaction design because a virtual pointer needs a way to distinguish pointing from drawing.

7.1.4 Interaction Characteristics

The five-sample moving average reduces small fluctuations in fingertip coordinates. The filter is deliberately simple and predictable. It does not estimate velocity or acceleration and does not adapt its smoothing strength to movement speed. Consequently, it provides basic stabilization while leaving room for more advanced filters in future versions.

The current interaction model uses one hand and a binary pinch state. This makes the system easier to understand and reduces the need for training data. At the same time, the limited state vocabulary constrains the number of distinct commands that can be represented naturally. The research paper identifies richer gesture vocabularies and learned classifiers as possible future improvements.

7.2 Discussion

The results support the feasibility of a lightweight webcam-based drawing interface. The functional screenshots show that hand movement can be converted into both continuous and discrete drawing actions. The pinch evaluation shows that the deterministic activation rule performed consistently on the available labelled samples. The strongest numerical characteristic is the absence of false positives in the evaluation set; the main observed error mode is missed pinch detection.

The system should nevertheless be evaluated within its stated scope. The project does not establish that MediaPipe or the AirSketch interaction model is universally accurate under arbitrary lighting, backgrounds, camera distances, users, or hand poses. The research paper deliberately avoids such a claim. Instead, the results show that the chosen configuration can support a practical drawing interaction under the tested conditions.

7.2.1 Comparison with Related Approaches

The research paper includes a limited numerical comparison with AO-Finger. AO-Finger reports 83% pinch precision in its sample-level evaluation, while AirSketch reports 100% precision on its own evaluation samples. The paper explicitly warns that these numbers are not a direct benchmark because the systems use different sensing modalities, datasets, participants, gesture definitions, sampling procedures, and evaluation protocols. The comparison therefore illustrates a difference in reported evaluation results rather than establishing superiority. From a system-design perspective, AirSketch trades sensing richness for simplicity. It uses a standard RGB webcam instead of dedicated acoustic-optic or depth hardware. This makes the system easier to reproduce and modify, while also making it more dependent on visual conditions.

7.3 TESTING AND EVALUATION

7.3.1 Testing Strategy

Testing was organized around two complementary goals. Functional testing checks whether the user-facing drawing and interface features respond to the intended hand interactions. Quantitative evaluation checks the pinch detector using labelled samples.

7.3.2 Pinch Detection Evaluation

The research paper reports an evaluation of 293 samples. The confusion matrix fig 7.3.2.1 contains 137 true negatives, 148 true positives, zero false positives, and eight false negatives. The evaluator samples every third frame when a ground-truth label has been selected and a hand is detected. This procedure is important because the reported result is a frame-level evaluation under the project's own labelling setup, not a participant-independent benchmark.

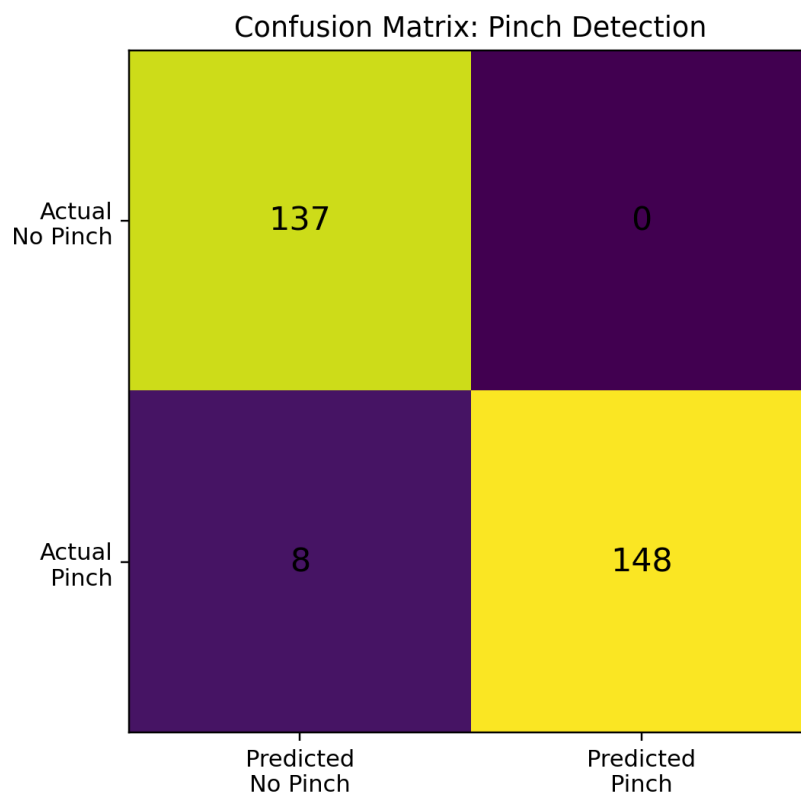


Figure 7.3.2.1: Confusion matrix for the reported 293-sample pinch evaluation.

7.3.3 Evaluation Methodology

Evaluation mode is toggled from the main application. The evaluator stores the current ground-truth label as either 0 or 1. A sample is considered only when a ground-truth label is present, a hand is detected, and the sampling interval is reached. The predicted value is the Boolean pinch state returned by HandTracker. Each accepted sample is written to a CSV file together with a timestamp and fingertip distance.

The use of explicit ground-truth labelling makes the experiment reproducible at the software level, but it also means that the quality of the result depends on the labelling procedure and test conditions. The current evaluation does not report the number of participants, lighting categories, camera models, or independent test sessions. Consequently, the numerical metrics should be interpreted as evidence about the available evaluation samples rather than a general population estimate.

7.3.4 Evaluation Metrics

Metric	Value	Interpretation for AirSketch
Samples	293	Number of labelled pinch/no-pinch samples used
Accuracy	97.27%	285 of 293 samples were classified correctly
Precision	100.00%	No false-positive pinch was observed
Recall	94.87%	8 of 156 actual pinch samples were missed
F1-score	97.37%	Harmonic mean of precision and recall
FPR	0.00%	No no-pinch sample was predicted as pinch
FNR	5.13%	8 actual pinch samples were predicted as no pinch

Table 7.3.4.1 Evaluation Metrics

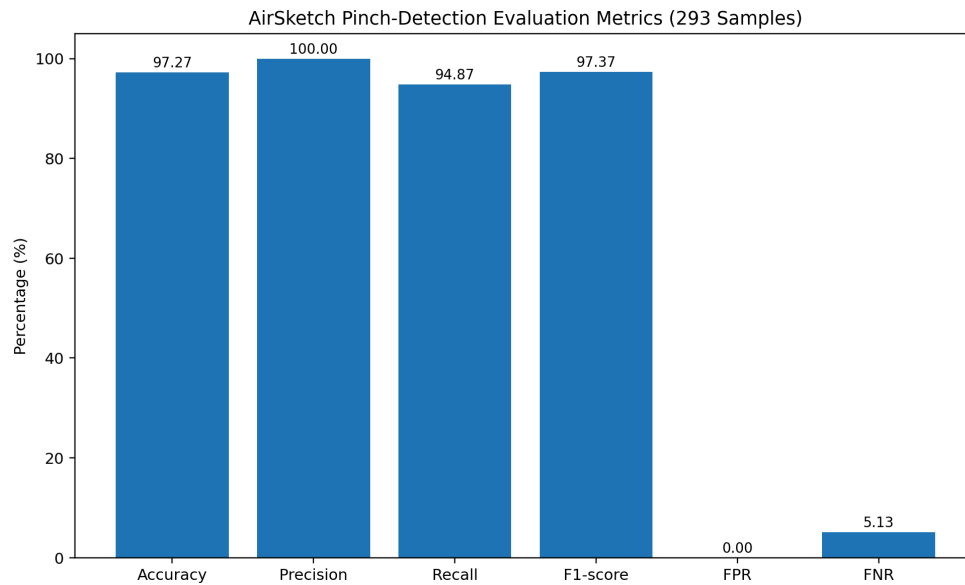


Figure 7.3.4.2: AirSketch pinch-detection evaluation metrics for 293 samples.

From Table 7.3.4.1 and its respective graphical representation shown in Figure 7.3.4.2, the reported accuracy of 97.27% means that 285 of the 293 labelled samples were classified correctly. Precision is 100%, because all 148 predicted pinch samples were true pinch samples and there were no false positives. Recall is lower at 94.87% because eight actual pinch samples were missed. The false-negative rate of 5.13% therefore represents the complete set of observed classification errors in the available evaluation.

From the interaction perspective, false positives and false negatives have different effects. A false-positive pinch could cause an unintended drawing action, while a false negative can prevent an intended stroke from starting or continuing. The current evaluation observed no false positives, but eight false negatives. Future threshold adaptation or temporal stabilization could therefore focus on reducing missed pinch states while preserving the low false-positive rate.

VIII. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

AirSketch demonstrates a complete real-time virtual drawing interaction using a conventional RGB webcam, MediaPipe hand landmarks, OpenCV, and NumPy. The project does not attempt to create a new hand-tracking model. Instead, it integrates established landmark detection with a deterministic pinch-based interaction model and a multi-tool drawing interface.

The system uses the index fingertip as the virtual pointer and the distance between the index and middle fingertips as the activation signal. A 70-pixel threshold determines the pinch state, while a five-sample moving average reduces fingertip jitter. The drawing engine supports brush, eraser, line, rectangle, circle, filled rectangle, filled circle, and clear operations. The UI provides twelve colours and ten brush-size levels.

The quantitative evaluation reported in the research paper contains 293 samples and produces 97.27% accuracy, 100.00% precision, 94.87% recall, and 97.37% F1-score. The confusion matrix contains 148 true positives, 137 true negatives, zero false positives, and eight false negatives. These results demonstrate consistent pinch detection on the available evaluation set, while the absence of environmental and multi-user benchmarking means that they should not be generalized beyond the tested conditions.

8.2 Current Limitations

- Only one detected hand is processed.
- The working coordinate space is fixed at 1280×720.
- The pinch threshold is fixed at 70 pixels and is not normalized by hand scale.
- The system depends on camera visibility and can be affected by lighting, background clutter, motion blur, and occlusion.

- The gesture vocabulary is small because no learned gesture classifier is used.
- The current evaluation focuses on pinch detection rather than comprehensive drawing quality or end-to-end user experience.
- Latency and frame-rate measurements are not included in the supplied evaluation results.

8.3 Future Enhancements

Future versions of AirSketch could introduce **multi-hand interaction**, scale-normalized pinch detection, and adaptive smoothing to improve interaction accuracy and drawing stability. Additional gestures could also be added for functions such as undo, redo, zoom, and tool switching. The system could further be evaluated under different users, lighting conditions, backgrounds, and camera distances to assess its robustness. Finally, **save and export functionality** could be incorporated, allowing users to store and reopen their drawings. These enhancements would improve the flexibility, consistency, and practical usability of the proposed gesture-based virtual drawing system.

8.4 Applications

- Educational demonstrations of computer vision and human–computer interaction.
- Touchless creative sketching and rapid visual ideation.
- Interactive classroom or presentation tools.
- Experimental accessibility-oriented interfaces.
- Gesture-based entertainment and interactive installations.
- Prototype environments for studying hand gesture interaction.
- Future integration with handwriting recognition or AI-assisted drawing systems.

REFERENCES

[1] C. Xu, B. Zhou, G. Krishnan, and S. Nayar, “AO-Finger: Hands-free Fine-grained Finger Gesture Recognition via Acoustic-Optic Sensor Fusing”, 2023.

<https://dl.acm.org/doi/10.1145/3544548.3581264>

[2] H. Chang, P. Ren, B. Zhang, D. Li, T. Han, H. Zhang, L. Xie, H. Chen, and E. Yin, “OMG-Bench: A New Challenging Benchmark for Skeleton-based Online Micro Hand Gesture Recognition,” 2026.

<https://arxiv.org/html/2512.16727v2>

[3] H. Jand and S. Ranjan, “Air Writing App with Real-Time Hand Tracking and Handwriting Recognition using Fingertip Detection and Drawing,” 2023.

researchgate.net/Air_Writing_App_with_Real-Time_Hand_Tracking_and_Handwriting_Recognition_using_Fingertip_Detection_and_Drawing

[4] S. A. Swathi, “Air Canvas Hand Tracking,” 2024.

https://www.jetir.org/view?paper=JETIR2411643&utm_source=chatgpt.com

[5] H. Mahmud, M. M. Morshed, and Md. Kamrul Hasan, “A Deep Learning-based Multimodal Depth-Aware Dynamic Hand Gesture Recognition System,” 2021.

https://www.researchgate.net/publication/366865288_Quantized_depth_image_and_skeleton-based_multimodal_dynamic_hand_gesture_recognition

[6] A. Sharma, A. Mittal, S. Singh, and V. Awatramani, “Hand Gesture Recognition using Image Processing and Feature Extraction Techniques”, 2020.

<https://www.sciencedirect.com/science/article/pii/S187705092031526X>

[7] J. Qi, L. Ma, Z. Cui, and Y. Yu, “Computer vision-based hand gesture recognition for human-robot interaction: a review,” 2024.

https://www.researchgate.net/publication/373049319_Computer_vision-based_hand_gesture_recognition_for_human-robot_interaction_a_review

[8] A. Morajkar, A. M. James, M. Bagwe, A. S. James, and A. Pavate, “Hand gesture and voice-controlled mouse for physically challenged using computer vision,” 2023.

https://www.researchgate.net/publication/370684551_Hand_gesture_and_voice-controlled_mouse_for_physically_challenged_using_computer_vision

[9] N. Anithadevi, S. Palanisamy, S. S. Rubini, and S. Shrestha, “MediaPipe-LSTM-Enhanced Framework for Real-Time Dynamic Sign Language Recognition in Inclusive Communication Systems,” 2025.

ResearchGatePublication

[10] L. Quan, Q. Guo, X. Xie, S. Chen, X. Li, and Y. Liu, “Towards Understanding the Faults of JavaScript-Based Deep Learning Systems”, 2022.

<https://dl.acm.org/doi/10.1145/3551349.3560427>

[11] R. Vasavi, N. Rahul, A. Snigdha, K. J. Moses, and S. V. Simha, “Painting with Hand Gestures using MediaPipe”, 2022.

<https://www.ijisrt.com/painting-with-hand-gestures-using-mediapipe>

[12] J. Moreira, “Dynamic LIBRAS Gesture Recognition via CNN over Spatiotemporal Matrix Representation”, 2026.

ResearchGatePublication

[13] H. X. G. Lim, X. Cui, Y. S. Rawat, and S.-N. Lim, “AirSketch: Generative Motion to Sketch,” 2024.

<https://arxiv.org/html/2407.08906v1>

[14] S. Barade, P. Kadam, P. Vedpathak, and N. S. Bhirame, “Virtual Write and Clear Application: A Gesture-based Drawing Tool using Mediapipe and OpenCV”, 2025.

https://www.ijmrset.com/upload/129_Virtual%20Write.pdf?utm_source=chatgpt.com

[15] S. V. Tiwari, D. A. Vartak, V. R. Verma, A. I. Sayyed, and S. Khatu, “Virtual Hand Gesture Painting,” 2024.

https://www.researchgate.net/publication/373559208_Virtual_Hand_Gesture_Painting

[16] Code-Hish, “MajorProject: New Airsketch Codes,” GitHub repository, 2026. Available:

<https://github.com/Code-Hish/MajorProject>